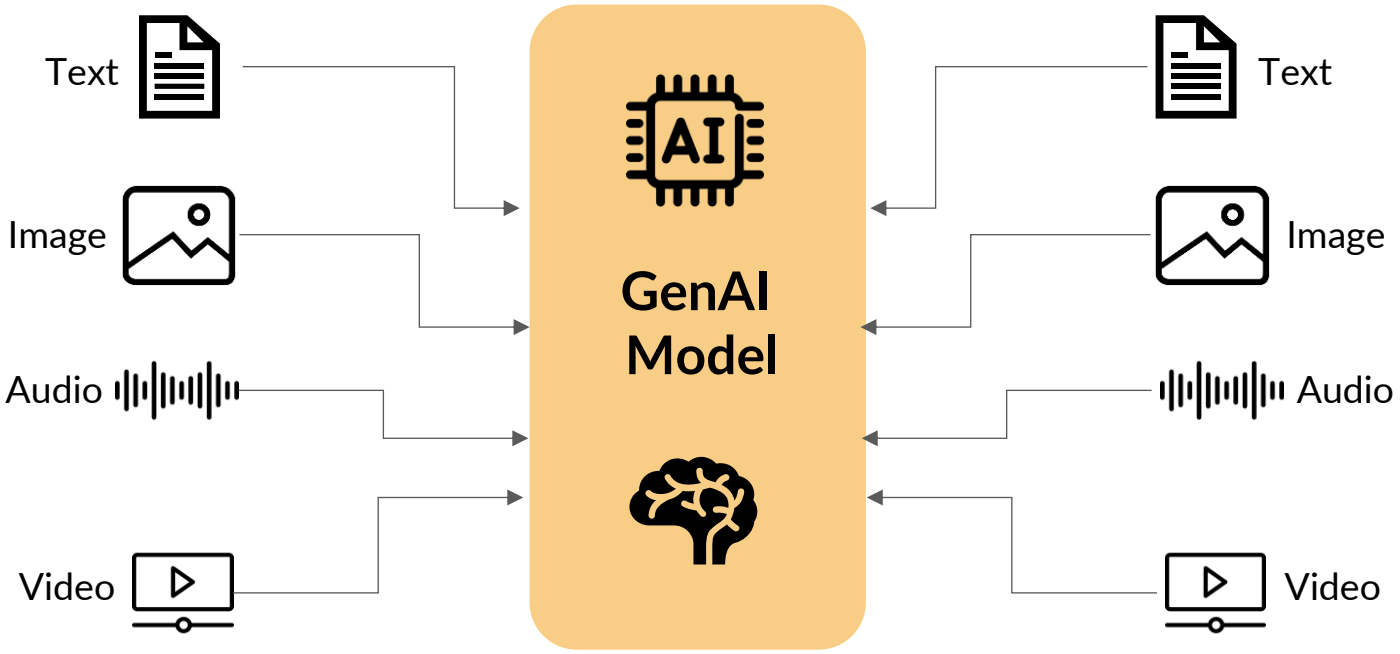
SESSION 3:
Multimodal & Model Landscape

Segment 1: Multimodal Prompting

UNDERSTANDING MULTIMODAL GENAI

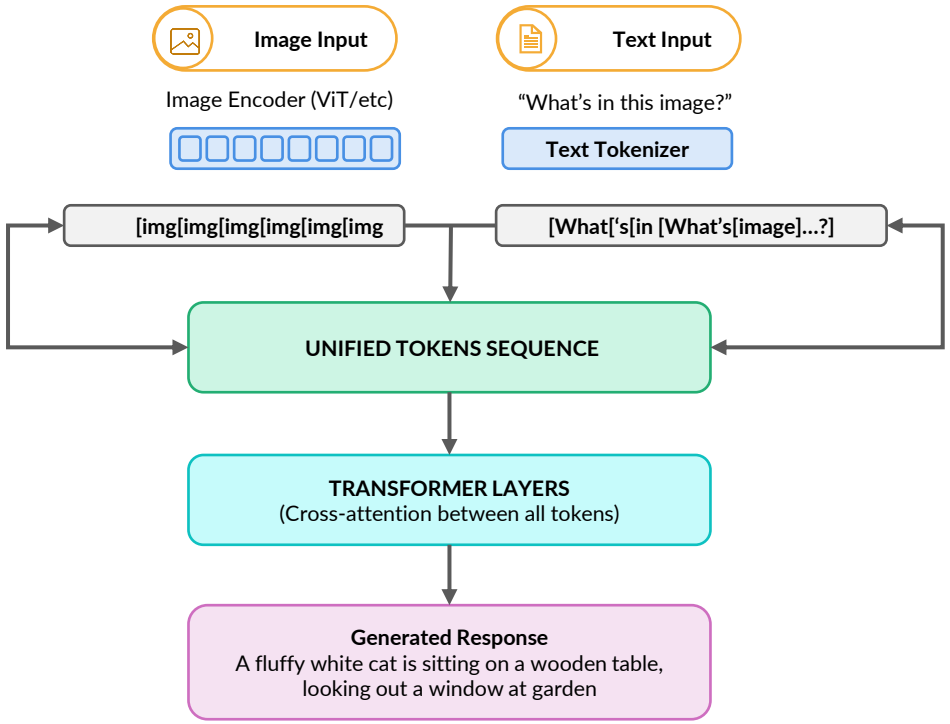
- Multimodal AI systems process and generate content across multiple modalities: text, images, audio, video, and documents
- This represents a fundamental expansion from text-only models, enabling AI systems to perceive and reason about the world in ways that more closely approximate human understanding



TEXT + IMAGE PROMPTING

Modern multimodal models (GPT-4V, Claude 3, Gemini) process images by

- **Encoding images into tokens:** Images are divided into patches and converted into embeddings that exist in the same vector space as text tokens
- **Interleaving with text:** Image tokens are positioned within the text sequence, allowing cross-attention between modalities
- **Unified reasoning:** The model reasons over both modalities jointly, not separately



PROMPTING STRATEGIES FOR TEXT + IMAGE

1. Direct Question: Best for simple identification, description tasks

```
[Image: Photo of a circuit board]
Prompt: "What components can you identify on this circuit board?"
```

2. Analytical Prompt: Best for data interpretation, trend analysis

```
[Image: Chart or graph]
Prompt: "Analyse this chart. What trends do you observe? What conclusions
can be drawn from the data?"
```

3. Comparative Prompt: Best for difference detection, progress tracking

```
[Image 1: Before photo]
[Image 2: After photo]
Prompt: "Compare these two images. What changes occurred between them?"
```

PROMPTING STRATEGIES FOR TEXT + IMAGE

4. Instructional Context: Best for expert analysis, professional review

```
[Image: Screenshot of UI]
Prompt: "You are a UX expert. Review this interface design and
identify usability issues. Prioritise by severity."
```

5. Grounded Generation: Best for content creation grounded in visual evidence

```
[Image: Product photo]
Prompt: "Write a compelling product description for an e-commerce
listing based on this image. Include key features you can observe."
```

6. Sequential Reasoning: Best for reasoning about a sequence of events

```
[Image 1] [Image 2] [Image 3] [Image 4]
Prompt: "These images show a process in order. Explain each step
and what's happening in the transition between steps."
```

BEYOND IMAGES: OTHER MODALITIES

Document Understanding (PDFs, Word Docs, Slides)

Documents are a distinct modality from simple images because they combine

- **Visual layout** (where things are positioned)
- **Textual content** (what the words say)
- **Structural semantics** (headers, tables, lists, footnotes)
- **Multi-page context** (relationships across pages)



Document-Specific Behaviours

Document Type	Strengths	Challenges
PDFs (text-based)	Good text extraction	Complex layouts confuse ordering
PDFs (scanned)	Can read with OCR	Quality depends on scan quality
Spreadsheets	Understands tabular data	Very large sheets exceed context
Presentations	Good at summarising slides	May miss speaker notes
Handwritten docs	Improving rapidly	Still struggles with poor handwriting
Forms	Good at field extraction	Nested/complex forms challenging

BEYOND IMAGES: OTHER MODALITIES

Speech and Audio

Audio understanding adds another dimension

- Temporal sequences
- Tone
- Speaker identity
- Paralinguistic features



Audio-Specific Behaviours

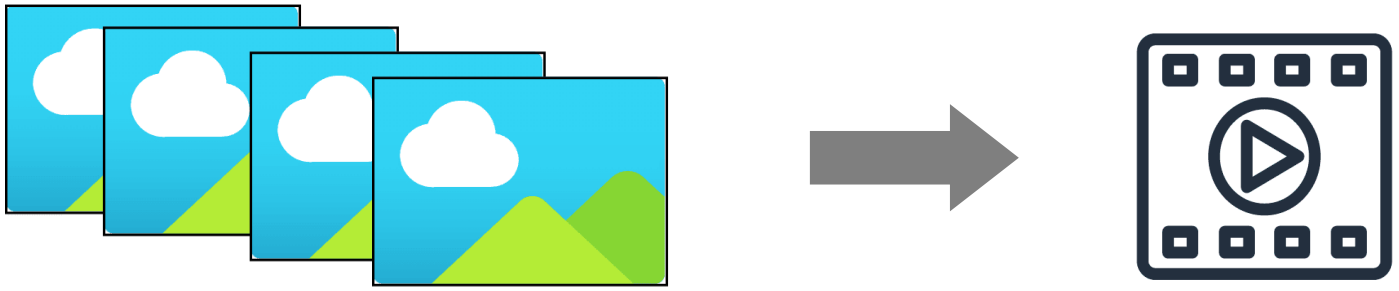
Capability	Current State	Limitations
Transcription	Excellent (Whisper-level)	Accents, overlapping speech
Speaker Diarisation	Good	Similar voices can confuse
Emotion Detection	Moderate	Cultural variation in expression
Music Understanding	Emerging	Limited to descriptions, not analysis
Sound Event Detection	Good for common sounds	Rare sounds may not be recognised
Language Detection	Excellent	Code-switching can confuse
Real-time Processing	Improving	Latency vs accuracy tradeoff

BEYOND IMAGES: OTHER MODALITIES

Video Understanding

Video combines the challenges of images, audio, and adds temporal reasoning. It comes with additional challenges such as

- Long videos exceed context limits
- Must sample frames intelligently
- Action recognition requires temporal reasoning
- Audio-visual alignment matters



Frames

Video

CURRENT CHALLENGES WITH MULTIMODAL GEN AI

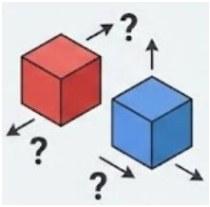
Counting

- How many people are in this crowd?
- Models often miscount, especially for > 10 items



Spatial relationships

- Is the red box to the left or right of the blue box?
- Can confuse left/right, above/below?



Fine text / Small details

- What does the small print say?
- May hallucinate or miss low-resolution text



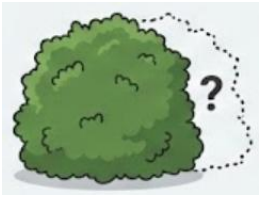
Temporal inference

- What season is this photo from?
- Inferences may be wrong or overconfident



Hidden / Occluded objects

- What's behind this object?
- Cannot see what's not visible (but may guess)



Faces and identity

- Who is this person?
- Often refuses or errs on side of caution



IDENTITY UNKNOWN /RESTRICTED

TYPES OF HALLUCINATION

Multimodal systems introduce new categories of hallucination and misalignment that don't exist in text-only models



Image shows: A park with trees and a bench

Model claims: "There's a dog sitting on the bench"

Reality: No dog exists in the image

Cause: Model's prior knowledge that parks often have dogs

Type 1: Object Hallucination



Image shows: A red car

Model claims: "The blue car is parked on the street"

Reality: Car exists but color is wrong

Cause: Weak binding between object and attribute

Type 2: Attribute Hallucination



Image shows: A man standing next to a woman

Model claims: "The man is hugging the woman"

Reality: They are just standing nearby

Cause: Inferring relationships not visually evident

Type 3: Relationship Hallucination



Document shows: Revenue: $1,234,567

Model claims: "Revenue: $1,234,657"

Reality: Digits are transposed

Cause: OCR errors, especially with similar-looking characters

Type 4: Text/ OCR Hallucination



Video shows: Person walks in, then sits down

Model claims: "Person sits down, then walks in"

Reality: Temporal order is reversed

Cause: Frame sampling loses temporal information

Type 5: Temporal Hallucination (Video/Audio)

Audio says: "The project is going well"



Image says: Team members looking frustrated

Model claims: "The team is happy with progress"

Reality: Visual audio signals conflict; model picks one

Cause: Frame sampling loses temporal information

Type 6: Cross-modal Hallucination

WHY MULTIMODAL HALLUCINATION OCCURS

01 Training Data Biases

Training data biases	Result
Photo of kitchens often contain cats	Model "sees" cats in kitchens even when absent
Photos of offices often contain computers	Model assumes computers exist in office photos
Photos of beaches often contain umbrellas	Model may add umbrellas to beach descriptions

HALLUCINATION

02 Resolution and Detail Loss

Image Resolution → Hallucination Risk

Original image: 4000 x 3000 pixels

Resized for Model: 512 x 512 pixels(typical)

Information lost

- Fine text becomes unreadable
- Small objects become ambiguous blobs
- Facial details are lost
- Textures become uniform

Model "files in" missing details form training priors → HALLUCINATION

03 Language Prior Dominance

[Empty Plate] + "Describe this meal" → "Delicious pasta with tomato sauce"

Why? The word "meal" activates food-related language patterns that override the visual evidence of an empty plate

04 Context Window Limitations

100-page Document

SUM

Page 1-10 Seen in detail

Page 11-90 Summarised or sampled

Page 91-100 Seen in detail

Risk: Model may hallucinate content for pages 11-90 or misattribute across pages

MODALITY MISALIGNMENT

Beyond hallucination, multimodal systems can exhibit misalignment between modalities

Common Modality Misalignments

Type	Description	Example
Grounding Failure	Text refers to non-existent visual elements	“Click the blue button” (no blue button exists)
Reference Ambiguity	Unclear which visual element text refers to	“This one”, when multiple items shown
Temporal Drift	Audio/video descriptions mismatch timing	Describing frame 1 when discussing frame 10
Modality Contradiction	Different modalities provide conflicting info	Image shows rain, text says “sunny day”
Context Leakage	Prior context influences unrelated analysis	Previous image affects current image description

COMMON MITIGATION STRATEGIES

1. Prompt Engineering for Reduced Hallucination

Explicit Grounding Instructions

```
Prompt: "Describe ONLY what you can directly observe in this image.
Do not infer or assume anything not visible.
If you're uncertain about something, say so.
Use phrases like 'appears to be' for uncertain observations."
```

Structured Output Requirements

```
Prompt: "For each claim about this image, rate your confidence:
-  HIGH: Clearly visible, unambiguous
-  MEDIUM: Partially visible or requires interpretation
-  LOW: Inferred from context, not directly visible

Format: [CLAIM] (CONFIDENCE: X)"
```

Negative Prompting

```
Prompt: “List what you see in this image.
Important: Do NOT mention objects that aren’t clearly visible.
Do NOT infer the presence of objects based on context.
Do NOT describe what ‘might’ be there."
```

COMMON MITIGATION STRATEGIES

2. Verification Strategies

- **Ask for Evidence**

```
Prompt: "You claimed there is a cat in this image.
Point to the specific region where the cat is located.
Describe the visual features that confirm it's a cat."
```

- **Cross-Validation**

```
Step 1: "List all objects in this image"
Step 2: "For each object, describe its exact location"
Step 3: "Now, look at the image again. Verify each object exists"
```

- **Ensemble Verification**

```
# Use multiple model calls with temperature variation
descriptions = []
for temp in [0.0, 0.3, 0.7]:
    descriptions.append(model.describe_image(image, temperature=temp))

# Objects mentioned in all 3 descriptions are more reliable
reliable_objects = intersection(descriptions)
```

COMMON MITIGATION STRATEGIES

3. System-Level Safeguards

AI Model Pipeline: From Input to Presentation

Input Stage

- Preserve high resolution when possible
- Provide multiple views/angles if available
- Include relevant metadata (source, date)

Processing Stage

- Use grounding-focused prompts
- Request confidence levels
- Ask for spatial references

Output Verification

- Cross-checking claims against image
- Flag low-confidence assertions
- Use specialised models for validation (e.g., object detection to verity claims)

User Presentation

- Show confidence indicators
- Link claims to image regions when possible
- Provide “verify” option for critical claims

COMMON MITIGATION STRATEGIES

4. Modality-Specific Verification

Modality	Verification Approach
Images	Object detection models, bounding box verification
Documents	OCR cross-check, layout analysis
Audio	ASR confidence scores, speaker verification
Video	Frame-by-frame validation, temporal consistency

Segment 2: Model Landscape

PROPRIETARY VS OPEN-SOURCE LLMs

The LLM landscape is fundamentally divided into two camps: open-source and proprietary models. Understanding this distinction is crucial for anyone looking to leverage AI in their applications

Proprietary LLMs

- Proprietary (or closed-source) LLMs are developed and controlled by companies that keep their model weights, training data, and architecture details private
- Users access these models through **APIs or provider-hosted services**
- **Examples:** GPT-4.1 (OpenAI), Claude 4 (Anthropic), Gemini 2.5 (Google)

ChatGPT Claude Gemini

Open-Source LLMs

- Open-source LLMs release their model weights publicly, allowing anyone to download, run, modify, and deploy them independently
- Users access these models through **locally-hosting the model** (using weights from the provider) or through **provider-hosted** services
- **Examples:** LLaMA 4 (Meta), Mistral Medium 7B (Mistral AI), Falcon, Qwen

LLaMA MISTRAL AI_ Qwen

PROPRIETARY LLMs

Benefits

- State-of-the-art performance on most benchmarks
- Managed infrastructure and scalability
- Regular updates and improvements
- Enterprise support and SLAs
- Built-in safety guardrails

Trade-offs

- Recurring API costs that scale with usage
- Data privacy concerns (your prompts go to external servers)
- Vendor lock-in and dependency
- Limited customisation (no access to model weights or architecture)
- No control over model changes or deprecation

OPEN-SOURCE LLMs

Benefits

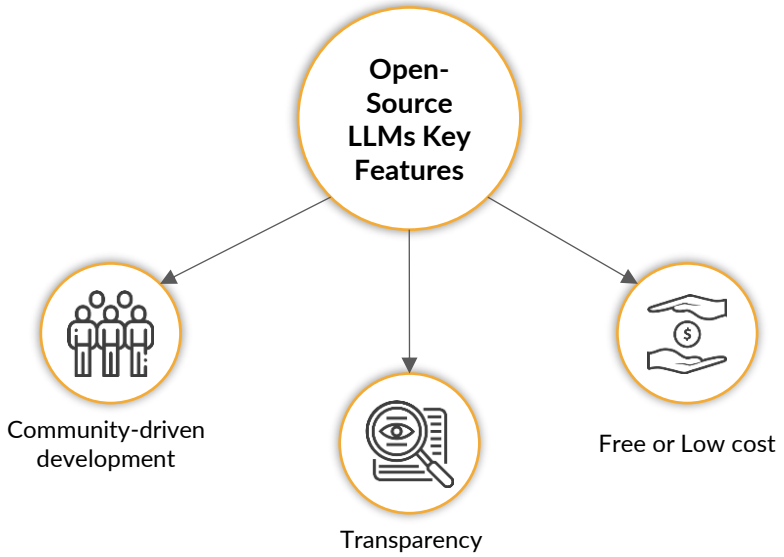
- Full control over data, nothing leaves your servers
- One-time infrastructure cost vs per-token pricing
- Complete customisation through fine-tuning
- No vendor dependency
- Greater transparency into model architecture and weights

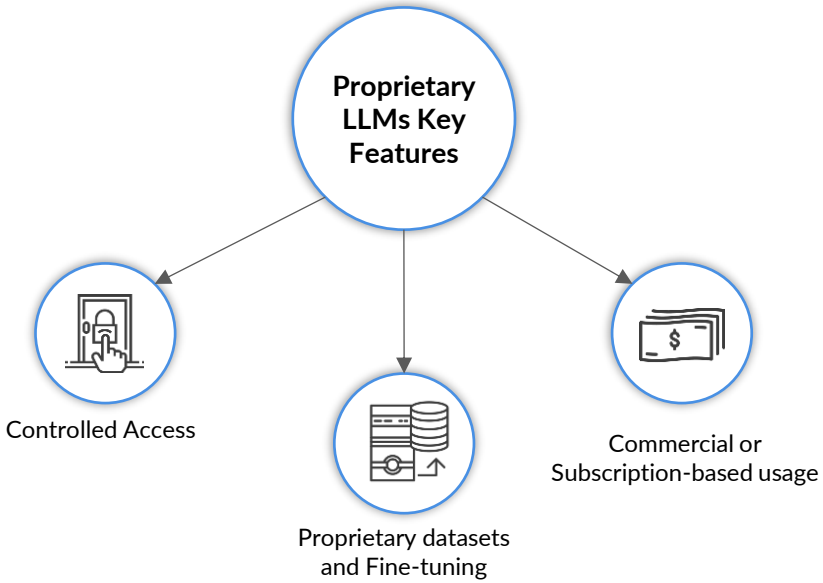
Trade-offs

- Requires significant hardware (GPUs) for inference
- Self-managed infrastructure and scaling
- May lag behind proprietary models in raw capability
- Security and safety are your responsibility
- Technical expertise required for deployment

WHEN TO CHOOSE WHICH MODEL?

Use Case	Recommended Approach
Rapid prototyping	Proprietary APIs
Sensitive/regulated data	Open-source (self-hosted)
High-volume production	Open-source (cost-effective at scale)
Cutting-edge performance	Proprietary
Custom domain adaptation	Open-source (fine-tuning)





GPT MODELS (OPENAI)

- **Strengths:** Superior reasoning, advanced code generation, multi-modal capabilities, enhanced instruction-following
- **Context Window:** Up to 256K tokens
- **Best For:** Complex reasoning tasks, agentic workflows, professional applications
- **Access:** API-only (chat.openai.com, Azure OpenAI)

OpenAI Open Models (gpt-oss)

- OpenAI has also released open-weight reasoning models under Apache 2.0 licensing
  - **gpt-oss-120b** — Full-scale reasoning model (90.0 MMLU, 80.1 GPQA Diamond)
  - **gpt-oss-20b** — Efficient model for local deployment
- Features: Chain-of-thought reasoning, tool use, agentic task support, full fine-tuning capability



CLAUDE (ANTHROPIC)

Anthropic's Claude emphasises safety, helpfulness, and handling long documents

- **Strengths:** Nuanced instruction-following, document analysis, reduced hallucination, strong ethical guardrails, and extended thinking capabilities
- **Context Window:** Up to 200K tokens
- **Best For:** Document processing, enterprise applications, tasks requiring careful reasoning
- **Access:** API, Claude.ai, Amazon Bedrock, Google Cloud

Claude

ANTHROP\C

GEMINI (GOOGLE DEEPMIND)

Google's multimodal model with deep integration into Google's ecosystem

- **Strengths:** Native multimodal understanding (text, image, audio, video), massive context window, Google Search integration, strong reasoning
- **Context Window:** Up to 1M tokens (Gemini 2.5 Pro)
- **Best For:** Multi-modal applications, research, long-document analysis, Google Workspace integration
- **Access:** API, Google AI Studio, Vertex AI

Gemini

Google DeepMind

LLAMA (META)

Meta's latest open-weight model family, pushing the boundaries of what open-source can achieve

- **Strengths:** Open weights, excellent performance-to-size ratio, active community, multimodal support, fine-tuning friendly
- **Context Window:** Up to 128K tokens
- **Model Family:** Llama 4, Llama 3.3 (stable), Llama Guard 4 (safety), Llama Prompt Guard 2
- **Best For:** Self-hosted deployments, fine-tuning, research, privacy-sensitive applications
- **Access:** Open download (Hugging Face, Kaggle, cloud partners)



MISTRAL AI

The European challenger known for efficiency, multilingual capabilities, and openness

- **Strengths:** Excellent efficiency, strong multilingual support (incl. European languages), competitive performance, various size options
- **Context Window:** Up to 128K tokens
- **Model Family:** Mistral Large 2, Mistral Medium, Mistral Small, Codestral (code-focused)
- **Best For:** European deployments (GDPR), cost-effective inference, multilingual applications, code generation
- **Access:** API (La Plateforme), open weights for some variants, cloud partners

MISTRAL AI_

COMPARING POPULAR LLMs

Model	Provider	Open Weights	Max Context	Multimodal	Reasoning Models	Best Use Case
GPT-5.2	OpenAI	No	256K	Yes	o3, o4-mini	General-purpose, Agentic
gpt-oss	OpenAI	Yes (Apache 2.0)	128K	No	Built-in	Local deployment, Fine-tuning
Claude 4	Anthropic	No	200K	Yes	Extended thinking	Documents, Enterprise
Gemini 2.5	Google	No	1M	Yes	Yes	Multimodal, Research
Llama 4	Meta	Yes	128K	Yes	No	Self-hosted, Fine-tuning
Mistral Large 2	Mistral	Partial	128K	Yes	No	Efficiency, Multilingual

LICENSING

Understanding how you can access and deploy LLMs is crucial for building production systems

Proprietary/Commercial Licenses

Provider	License Type	Key Terms
OpenAI	Commercial API	Pay-per-token, usage policies apply, no model access
Anthropic	Commercial API	Pay-per-token, acceptable use policy
Google	Commercial API	Pay-per-token, integrated with GCP billing

Open-Source Licenses

Model	License	Commercial Use	Modifications	Key Restrictions
LLaMA 4	Meta License	Yes (with conditions)	Yes	Must accept license, usage restrictions for 700M+ MAU
Mistral	Apache 2.0	Yes	Yes	None (fully permissive)
Falcon	Apache 2.0	Yes	Yes	None
Qwen	Qwen License	Yes	Yes	Attribution required

ACCESS MECHANISM

01 Direct API Access




```
from openai import OpenAI

client = OpenAI()

response = client.chat.completions.create(
    model="gpt-4.1",
    messages=[{"role": "user", "content": "Hello"}],
)
```

- **Pros:** Simplest setup, always up-to-date
- **Cons:** Data leaves your infrastructure, ongoing costs

02 Cloud Platform APIs

Azure OpenAI	Amazon Bedrock	Google Vertex AI
GPT-4.1	GPT-4.1 GPT-4o	Gemini, PaLM, Open models
Azure OpenAI	Claude, LLaMA, Mistral, Titan	Gemini, PaLM, Open models

- **Pros**: Enterprise compliance, unified billing, VPC integration
- **Cons:** Additional layer, potential markup

03 Self-Hosted Deployment



Popular Serving Frameworks

- **VLLM:** High-throughput serving with PagedAttention
- **Text Generation Inference (TGI):** Hugging Face's production server
- **Ollama:** Easy local deployment for development

- **Pros:** Full data control, no per-token costs, customisation
- **Cons:** Requires GPU infrastructure, operational overhead

04 Managed Open-Source

Provider	Model Available
Together AI	LLaMA, Mistral, and 100+ models
Replicate	Various open models
HuggingFace	Any HuggingFace model
Anyscale	LLaMA, Mistral

- **Pros:** open model flexibility without infrastructure burden

DEPLOYMENT OPTIONS

Deployment Decision Matrix

Priority	Recommended Approach
Fastest time-to-market	Proprietary API
Maximum data privacy	Self-hosted open-source
Enterprise compliance	Cloud platform (Azure/AWS/GCP)
Cost optimisation at scale	Self-hosted or managed open-source
Cutting-edge capabilities	Proprietary API
Full customisation	Self-hosted + fine-tuned

Cost Considerations

Deployment Model	Cost Structure	Best For
Proprietary API	Per-token (input + output)	Low-medium volume, variable usage
Cloud Platform	Per-token + platform fees	Enterprise, compliance needs
Managed Open-Source	Per-token (usually cheaper)	Medium volume, want simplicity
Self-Hosted	Fixed infra + operational	High volume, predictable usage

Rule of Thumb: Self-hosting becomes cost-effective when your monthly API bill exceeds your infrastructure + operational costs, typically at high volumes (millions of tokens/day)

LOCAL LLMs

- Running Open Source LLMs locally offers significant advantages
  - Complete data privacy
  - No API costs
  - Offline access
  - Full customisation
- With tools like Ollama and increasingly efficient open-weight models, local LLM deployment is now accessible to developers with consumer-grade hardware

The Core Advantages of Local Deployment



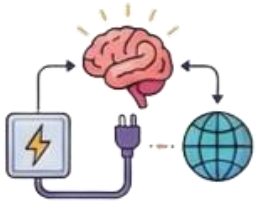
Complete Data Privacy

Ensure sensitive information never leaves your local hardware or internal network



Zero API and Token Costs

Eliminate recurring subscription fees and per-token usage charges for development



Offline Access & Control

Maintain full model functionality and deep customisation without an internet connection

OLLAMA

Ollama is the simplest way to run LLMs locally. It handles model downloading, optimisation, and serving with a single command-line tool

Ollama is an open-source tool that

- Downloads and manages LLM model files
- Optimises models for your hardware (CPU/GPU)
- Provides a simple CLI and REST API
- Handles quantisation and memory management automatically



LLAMA 4

LLaMA 4 is Meta's latest open-weight model family offers excellent performance across various sizes

Available Variants

Model	Parameters	Size	RAM Required	Best For
llama4:8b	8B	~4.7 GB	8 GB+	Fast responses, consumer hardware
llama4:70b	70B	~40 GB	48 GB+	High quality, workstations
llama4:8b-instruct	8B	~4.7 GB	8 GB+	Chat/instruction following

Strengths of LLaMA 4

- Excellent general knowledge
- Strong reasoning capabilities
- Good multilingual support
- Active community and fine-tunes
- Multimodal variants available



MISTRAL AI

Mistral AI's models are known for efficiency and a strong performance-to-size ratio

Available Variants

Model	Parameters	Size	RAM Required	Best For
mistral:7b	7B	~4.1 GB	8 GB+	Efficient general use
mistral:instruct	7B	~4.1 GB	8 GB+	Instruction following
mistral-large	123B	~70 GB	96 GB+	Maximum capability

Strengths of Mistral

- Excellent efficiency (great quality/size ratio)
- Strong European language support
- Fast inference speed
- Apache 2.0 license (most variants)
- Specialised code model (Codestral)

Mistral AI

DEEPSEEK

DeepSeek offers powerful models with strong reasoning capabilities, especially the R1 series

Available Variants

Model	Parameters	Size	RAM Required	Best For
deepseek-r1:8b	8B	~4.9 GB	8 GB+	Reasoning, consumer HW
deepseek-r1:14b	14B	~8.5 GB	16 GB+	Better reasoning
deepseek-r1:32b	32B	~19 GB	32 GB+	High-quality reasoning
deepseek-r1:70b	70B	~40 GB	48 GB+	Maximum capability
deepseek-coder	33B	~19 GB	32 GB+	Code generation

Strengths of DeepSeek R1

- Excellent reasoning (chain-of-thought)
- Strong mathematical abilities
- Competitive with proprietary models
- Shows reasoning process
- MIT license



GPT-OSS

GPT-OSS represents OpenAI's first open-weight models released under the Apache 2.0 license, allowing full commercial use, modification, and local deployment. These models are specifically designed for reasoning tasks and agentic workflows

Available Variants

Model	Parameters	Size	RAM Required	Best For
gpt-oss-20b	20B	~12 GB	16 GB+	Consumer hardware, balanced performance
gpt-oss-120b	120B	~70 GB	96 GB+	Maximum capability, enterprise/server
gpt-oss-safeguard	Various	Varies	Varies	Safety-focused deployments

Strengths of GPT-OSS

- **Reasoning:** Best-in-class for complex problem solving
- **Speed:** Exceptional throughput (3072 tok/s for 120B)
- **Instruction Following:** Excellent adherence to complex instructions
- **Agentic Tasks:** Native tool use and function calling
- **Code Generation:** Strong coding capabilities



LOCAL MODEL COMPARISON

Feature	GPT-OSS	LLaMA 4	Mistral	DeepSeek R1
License	Apache 2.0	Meta License	Apache 2.0	MIT
Best Size	20B/120B	8B/70B	7B/Large	8B/70B
Reasoning	5/5	4/5	3/5	5/5
Speed	5/5	4/5	5/5	3/5
Tool Use	Native	Via prompting	Via prompting	Native
Multi-modal	No	Yes	Yes	No

HOW TO SELECT LOCAL LLMs

Follow these guidelines if you need a local LLM deployment

LIMITED HARDWARE (8-16 GB RAM)

- Speed priority → **Mistral 7B**
- Reasoning priority → **DeepSeek R1 8B**
- General use → **LLaMA 4 8B**

MID-RANGE HARDWARE (32 GB RAM)

- OpenAI-style reasoning → **GPT-OSS 20B**
- Multi-modal needed → **LLaMA 4 8B**
- Code generation → **DeepSeek Coder**

ENTERPRISE HARDWARE (64 GB+ RAM)

- Maximum reasoning → **GPT-OSS 120B**
- Balanced performance → **LLaMA 4 70B**
- Deep analysis → **DeepSeek R1 70B**